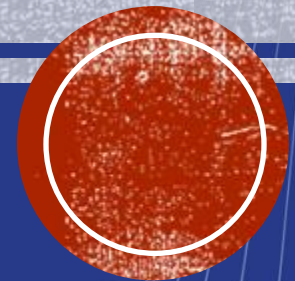




Mahidol University
Wisdom of the Land

WEEK 3

Probabilistic Approach

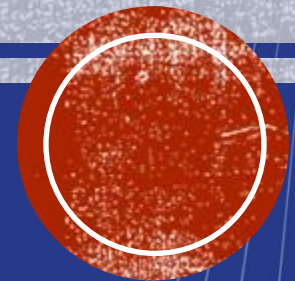




Mahidol University
Wisdom of the Land

WEEK 3

Continuous Approach



IF THE DATA IS NOT DISCRETE

- Some none existing data may not be able to estimate
- **NAN** values that show that data is NOT present in the training set
- Solution: Make a continuos data



THE MOST POPULAR DISTRIBUTION

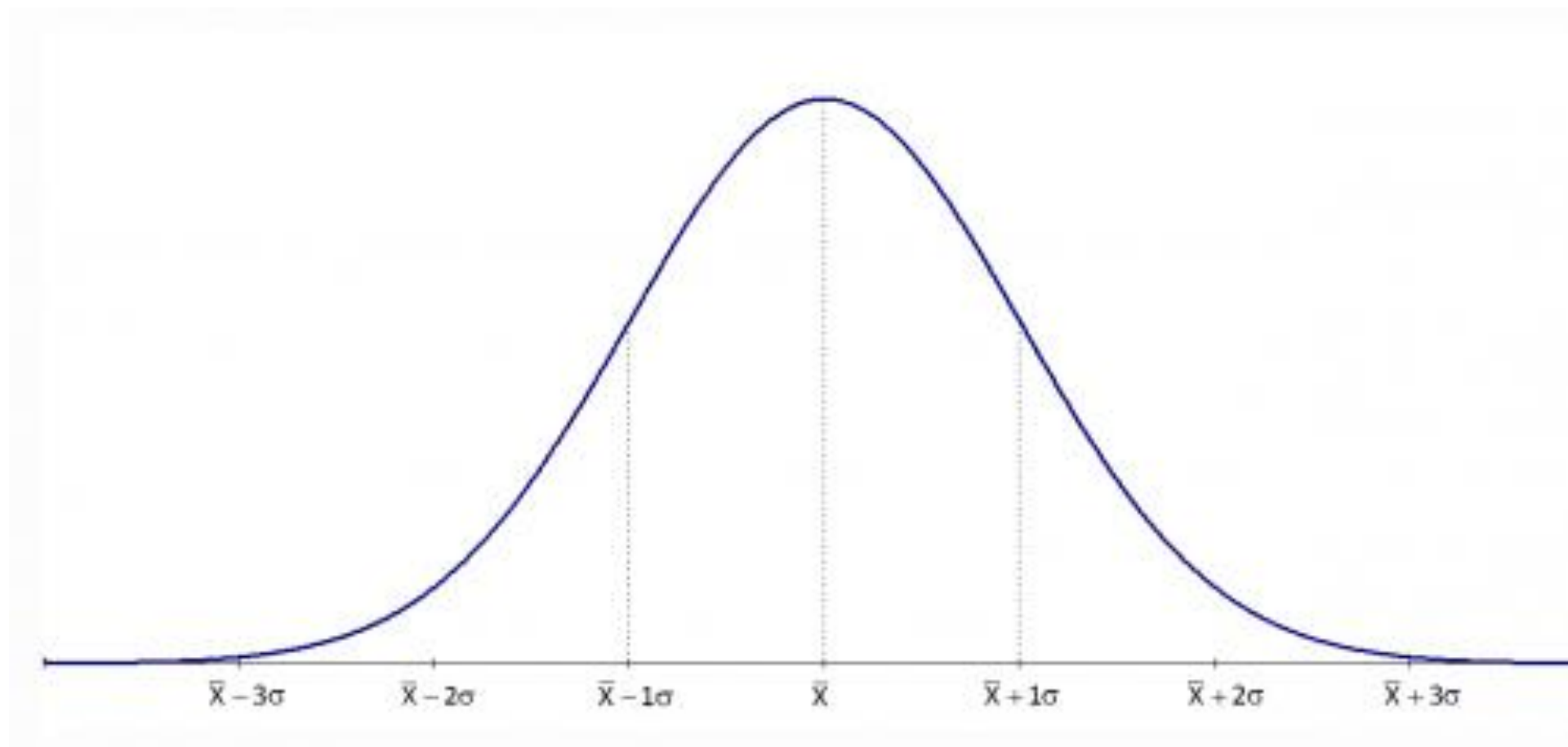
- Guassian Distribution
- Poisson Distribution
- Gamma Distribution



GAUSSIAN DISTRIBUTION

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

where σ is the standard deviation and μ the mean



python=

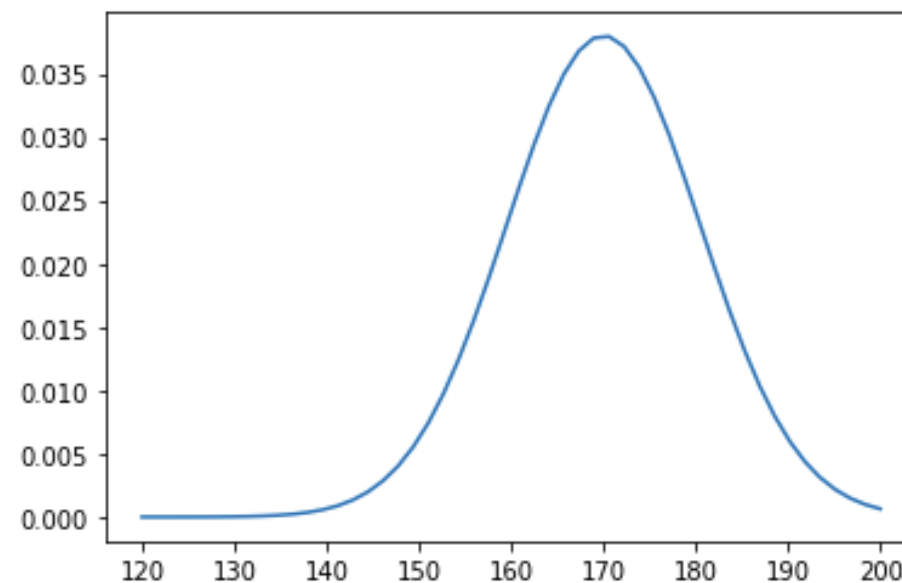
```
import scipy.stats as sc  
sc.norm(160,1.63)
```



GUASSIAN DISTRIBUTION(NORMAL DISTRIBUTION)

```
import matplotlib.pyplot as plt
    mean    std
plt.plot(x,sc.norm(170,10.5).pdf(x))
```

[<matplotlib.lines.Line2D at 0x7fb032fb0190>]



With mean=170 and sd=10.5
“probability distribution”



USE THE FOLLOWING FILE

- Weight Height M W.csv
- Try describe and print it out



TRANSFORM TO PROBABILITY?

HEIGHT	M	W
	160	160
	175	158
	165	162
AVG	166.6	160
SD	6.23	1.63

- If the height is 158, what is the probability of being **M/W**

$$P(W | 158) = P(158 | W) P(W) / P(158)$$

python script

```
import scipy.stats as sc
sc.norm(160, 1.63).pdf(158)*.5
/(sc.norm(160, 1.63).pdf(158)*0.5+0.5*sc.norm(166.6, 6.23).pdf(158))
```

$$P(M | 158) = P(158 | M) P(M) / P(158)$$



DETERMINE WHICH PROBABILITY

- $P(M | 158)$ and $P(W | 158)$
- $P(158)$ is usually omitted.

```
=====
Height
mean          168.2513 160.1316
std. dev.      3.1034  3.7677
weight sum      100    100
precision      0.1198  0.1198
```



PROBABILITY DISTRIBUTION

- $P(W | \text{DATA}) = \frac{P(\text{DATA} | W) P(W)}{P(\text{DATA})}$
- What is the data?
 - One dimensional
 - Weight only
 - Height only
 - Two dimensional
 - Weight & Height
 - Multidimensional
 - Weight & Height & many more



ONE DIMENSIONAL

IF my height=177 what gender am I?

$$P(W | DATA) = P(DATA | W) P(W) / P(DATA) \\ = P(DATA | W) P(W) / P(DATA)$$

With Guassian
distribution
find Mean and std of
all woman

prio

0.5

With Guassian
distribution
find Mean and std of
all data
or

Marginalized
distribution

```
mean_HW=df.loc[df['gender']=='W','Height'].mean()  
sd_HW =df.loc[df['gender']=='W','Height'].std()
```

```
sc.norm(mean_HW, sd_HW).pdf(177)
```



ONE DIMENSIONAL

IF my height=177 what gender am I?

$$P(M | DATA) = \frac{P(DATA | M) P(M)}{P(DATA)}$$
$$= \frac{P(DATA | M) P(M)}{P(DATA)}$$

With Guassian
distribution
find Mean and std of
all woman

prio

0.5

With Guassian
distribution
find Mean and std of
all data
or

Marginalized
distribution

```
mean_HM=df.loc[df['gender']=='M', 'Height'].mean()  
sd_HM =df.loc[df['gender']=='M', 'Height'].std()  
  
sc.norm(mean_HM, sd_HM).pdf(177)
```



ONE DIMENSIONAL

IF my height=177 what gender am I?

Marginalized
distribution

$$P(\text{DATA}) = P(\text{DATA} | W) P(W) + P(\text{DATA} | M) P(M)$$

```
mean_H=df.loc['Height'].mean()  
sd_H =df.loc['Height'].std()
```

```
sc.norm(mean_H,sd_H).pdf(177)
```

With Guassian
distribution
find Mean and std of
all data



TWO DIMENSIONAL PROBABILITY OF WOMAN

IF my height=177 weight=60, what gender am I?

$$P(W | \text{DATA}) = P(\text{DATA} | W) P(W) / P(\text{DATA})$$
$$= \text{P}(\text{DATA} | \text{W}) P(W) / P(\text{DATA})$$

With Guassian
distribution
find Mean and std of
all woman

prior

0.5

```
mean_WW=df.loc[df['gender']=='W','Weight'].mean()
```

```
sd_WW =df.loc[df['gender']=='W','Weight'].std()
```

```
sc.norm(mean_WW,sd_WW).pdf(60)*sc.norm(mean_HW,sd_HW).pdf(177)
```



TWO DIMENSIONAL PROBABILITY OF MALE

IF my height=177 weight=60, what gender am I?

$$P(M | \text{DATA}) = P(\text{DATA} | M) P(M) / P(\text{DATA})$$
$$= P(\text{DATA} | M) P(M) / P(\text{DATA})$$

With Guassian
distribution
find Mean and std of
all woman

prior

0.5

```
mean_WM=df.loc[df['gender']=='M','Weight'].mean()  
sd_WM =df.loc[df['gender']=='M','Weight'].std()
```

```
sc.norm(mean_WM,sd_WM).pdf(60)*sc.norm(mean_HM,sd_HM).pdf(177)
```



TWO DIMENSIONAL PROBABILITY

Given a set of data x

```
sc.norm(mean_WM, sd_WM).pdf(x['weight']) * sc.norm(mean_HM, sd_HM).pdf(x['height'])
```

```
P(M|DATA) = P(DATA|M) * P(M) / P(DATA)
            = sc.norm(mean_WM, sd_WM).pdf(x['weight'])
              * sc.norm(mean_HM, sd_HM).pdf(x['height'])
              * 0.5
            / P(DATA)
```



N - DIMENSIONAL TECHNIQUES

- Bayes' Bayes assumes Independence

$$p(c | f_1, \dots, f_n) \propto p(c) p(f_1 | c) \dots p(f_n | c)$$

- Example

$$P(W | 158, 60) \propto P(158 | W) P(60 | W) P(W)$$



TWO DIMENSIONAL PROBABILITY

Given a set of data x

```
sc.norm(mean_WM, sd_WM).pdf(x['weight']) * sc.norm(mean_HM, sd_HM).pdf(x['height'])
```

```
P(M|DATA) = P(DATA|M) * P(M) / P(DATA)
            = sc.norm(mean_WM, sd_WM).pdf(x['weight'])
              * sc.norm(mean_HM, sd_HM).pdf(x['height'])
              * 0.5
              / P(DATA)
```



EXERCISE II

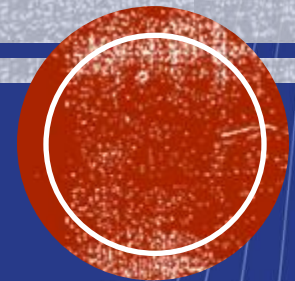




Mahidol University
Wisdom of the Land

WEEK 3

Integration with SKLearn



SKLEARN FOR DATA SPLITTING

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, Y_train, Y_test =  
    model_selection.train_test_split  
    (X, Y, test_size=test_size, random_state=seed)
```

Try

```
X = np.array([[1, 2], [3, 4], [1, 2], [3, 4], [1, 2], [3, 4]])
```

```
y = np.array([0, 0, 0, 1, 1, 1])
```



BAYES AS A SUPERVISED LEARNING

1. Choose your model

- For continuous

```
#load library for continuous (Gaussian)
from sklearn.naive_bayes import GaussianNB
classifier=GaussianNB()
```

- For Discretized

```
#load library for Discrete value
from sklearn.naive_bayes import MultinomialNB
classifier=MultinomialNB()
```



BAYES AS A SUPERVISED LEARNING

2. Train the model

- Set the prior

```
classifier.class_prior_ = [0.3, 0.7]
```

- Train(Find the likelihood)

```
Model=classifier.fit(feature, expected_output)
```

3. Understand your output

- Predict/(Find the highest Prob)

```
Model.predict(number)
```
- Give the probability output $P(\text{Class}|\text{Data})$
 - ```
Model.predict_log_proba(number)
```
- Give the accuracy a single prediction
  - ```
Model.score(number, digit_num[:,0])
```

